

Pumas Sangrientos

Team Reference Document

Índice

1	Basics	1
1.1	Standard Competitive Programming Template	1
2	Data Structures	1
3	Geometry	1
4	Graphs	1
5	Math	1
5.1	Binomial Coefficients Modulo Prime	1
6	Strings	1

1. Basics

1.1. Standard Competitive Programming Template

Autor: Pumas Sangrientos

Descripción: Plantilla base con optimización de I/O, macros comunes y typedefs esenciales para reducir tiempo de codificación.

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 // === Typedefs ===
6 typedef long long ll;
7 typedef long double ld;
8 typedef pair<int, int> pii;
9 typedef pair<ll, ll> pll;
10 typedef vector<int> vi;
11 typedef vector<ll> vll;
12
13 // === Macros ===
14 #define endl '\n'
15 #define pb push_back
16 #define all(x) (x).begin(), (x).end()
17 #define sz(x) (int)(x).size()
18 #define FOR(i, a, b) for(int i = (a); i < (b); ++i)
19 #define ROF(i, a, b) for(int i = (a); i >= (b); --i)
20
21 void fast_io() {
22     ios_base::sync_with_stdio(false);
23     cin.tie(NULL);
24     cout.tie(NULL);
25 }
26 }
```

```
27
28 void solve() {
29     // Escribe tu lógica aquí
30 }
31
32 int main() {
33     fast_io();
34
35     int t = 1;
36     // cin >> t; // Descomentar si el problema tiene múltiples casos de prueba
37     while(t--) {
38         solve();
39     }
40
41     return 0;
42 }
```

2. Data Structures

3. Geometry

4. Graphs

5. Math

5.1. Binomial Coefficients Modulo Prime

Autor: botellot

Descripción: Calcula coeficientes binomiales (nCr) usando factoriales precomputados e inversos modulares. Requiere que m sea primo.

```
1 const int MAXN = 1000005; // Ajustar según el problema
2 // 0 998244353
3 ll factorial[MAXN];
4 ll inv_factorial[MAXN];
5 // Exponenciación binaria para el inverso modular  $O(\log m)$ 
6 ll power(ll base, ll exp) {
7     ll res = 1;
8     base %= m;
9     while (exp > 0) {
10         if (exp % 2 == 1) res = (res * base) % m;
11         base = (base * base) % m;
12         exp /= 2;
13     }
14     return res;
15 }
16 ll modInverse(ll n) {
17     return power(n, m - 2);
18 }
19 // Precomputo en  $O(MAXN)$ 
20 void precompute() {
21     factorial[0] = 1;
22     for (int i = 1; i < MAXN; i++)
23         factorial[i] = (factorial[i - 1] * i) % m;
24     // Inverso del factorial final usando Fermat
25     inv_factorial[MAXN - 1] = modInverse(factorial[MAXN - 1]);
26 }
```

```
26 |         // Inversos de los demás factoriales en  $O(MAXN)$ 
27 |         for (int i = MAXN - 2; i >= 0; i--)
28 |             inv_factorial[i] = (inv_factorial[i + 1] * (i + 1)) % m;
29 |     }
30 |     // Cálculo en  $O(1)$ 
31 | ll nCr(int n, int k) {
32 |     if (k < 0 || k > n) return 0;
33 |     return factorial[n] * inv_factorial[k] % m * inv_factorial[n - k] % m;
34 | }
```

6. Strings